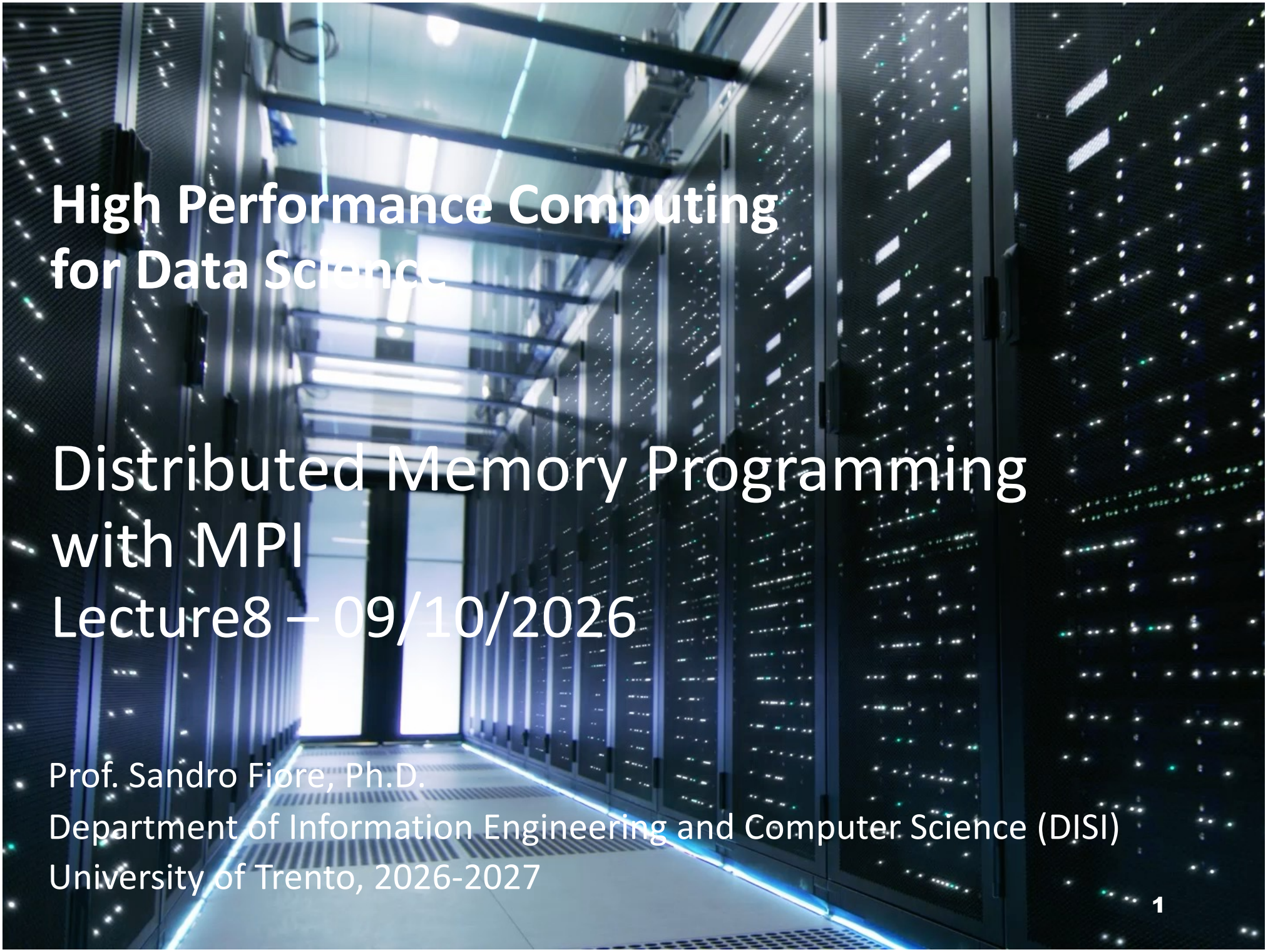


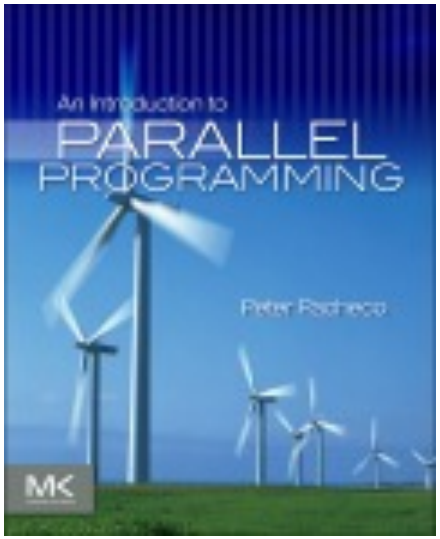
High Performance Computing for Data Science

Distributed Memory Programming with MPI
Lecture8 – 09/10/2026

Prof. Sandro Fiore, Ph.D.
Department of Information Engineering and Computer Science (DISI)
University of Trento, 2026-2027

Today's outline and material

- Demo + Job submission (sequential + parallel)
- Trapezoidal rule (second part)



Today's material

Book
An Introduction to Parallel Programming by Peter Pacheco

Chapter 3
Distributed Memory Programming with MPI

Login to the HPC cluster

- Turn on VPN
- Check with ping if the login node is reachable
- `sfiore$ ping hpc3-login.unitn.it`

  …

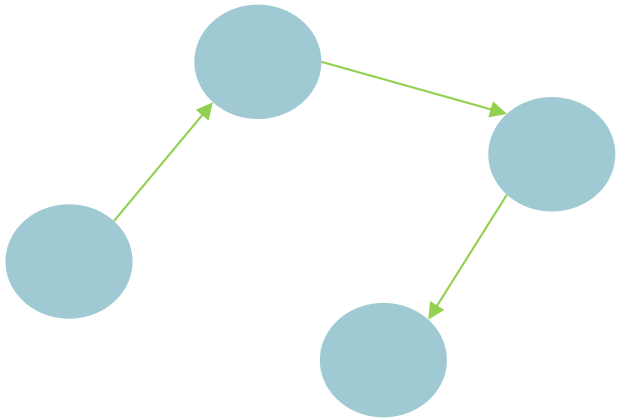
- Then run ssh using your ‘username’ e pwd (your UniTrento credentials):

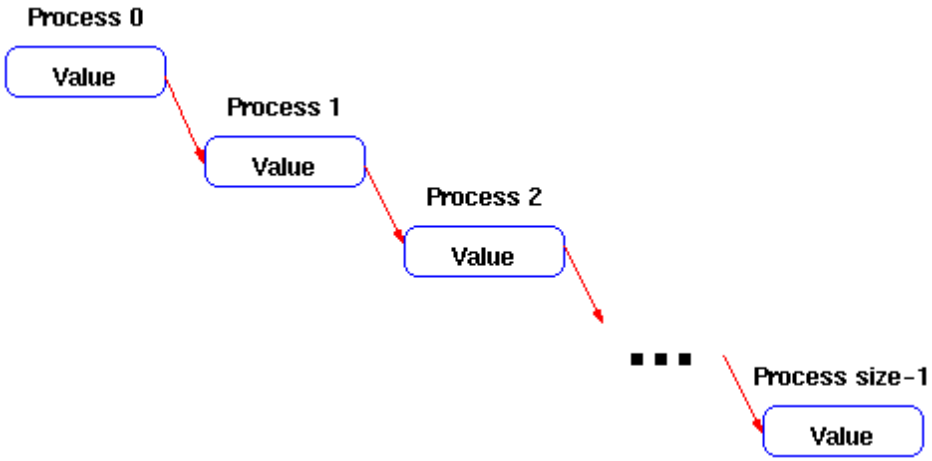
  `ssh sandro.fiore@hpc3-login.unitn.it`

Communication hands-on

Implement a ring by using send and receives

Write a program that takes data from process zero and sends it to all of the other processes by sending it in a ring. That is, process i should receive the data and send it to process i+1, until the last process is reached.





Communication hands-on (solution)

```
#include <stdio.h>
#include "mpi.h"

int main( argc, argv )
int argc;
char **argv;
{
    int rank, value, size;
    MPI_Status status;

    MPI_Init( &argc, &argv );

    MPI_Comm_rank( MPI_COMM_WORLD, &rank );
    MPI_Comm_size( MPI_COMM_WORLD, &size );
    do {
        if (rank == 0) {
            scanf( "%d", &value );
            MPI_Send( &value, 1, MPI_INT, rank + 1, 0, MPI_COMM_WORLD );
        }
        else {
            MPI_Recv( &value, 1, MPI_INT, rank - 1, 0, MPI_COMM_WORLD,
                      &status );
            if (rank < size - 1)
                MPI_Send( &value, 1, MPI_INT, rank + 1, 0, MPI_COMM_WORLD );
        }
        printf( "Process %d got %d\n", rank, value );
    } while (value >= 0);

    MPI_Finalize( );
    return 0;
}
```

Module environment

```
Module [list | avail | load | unload]
```

- *list*: it lists the modules already loaded
- *load* <module> : it loads a module
- *unload* <module>: it unloads a module
- *avail*: it lists the modules available on the cluster

- ***On our cluster MPI requires***
  - ***module load MPICH/3.4.2-GCC-10.3.0***

Basics of PBS CLI (I)

Sequential and parallel job submission

- To submit a job: **qsub** <*script*>
  - *You get back a job_ID*
- To monitor a job: **qstat** <*job_ID*>
  - *You get back job status info*
- To cancel a job: **qdel** <*job_ID*>
  - *Action job cancelled*

Basics of PBS CLI (II)
Job submission (sequential job)

[sandro.fiore@hpc3-login0 ~]$ more test1.sh

```
#!/bin/bash

#PBS -l select=1:ncpus=1:mpiprocs=1:mem=2gb

# set max execution time
#PBS -l walltime=0:00:30

# set the queue
#PBS -q shortCPUQ

echo "hostname: " && hostname
```

[sandro.fiore@hpc3-login0 ~]$ qsub test1.sh

[sandro.fiore@hpc3-login0 ~]$ qstat <ID>

Basics of PBS CLI (III)
Job submission (paralle job – single node)

[sandro.fiore@hpc3-login0 ~]$ more test2.sh

```
#!/bin/bash

#PBS -l select=1:ncpus=4:mem=2gb

# set max execution time
#PBS -l walltime=0:00:30

# set the queue
#PBS -q shortCPUQ

module load MPICH/3.4.2-GCC-10.3.0
mpirun -n 4  ./ring
```

[sandro.fiore@hpc3-login0 ~]$ qsub test2.sh

[sandro.fiore@hpc3-login0 ~]$ qstat <ID>

Homework - Communication

Design a broadcast solution by using send and receive calls

What we expect: a process “p” wants to share the content of a variable “a” with all the other processes.

Question ?

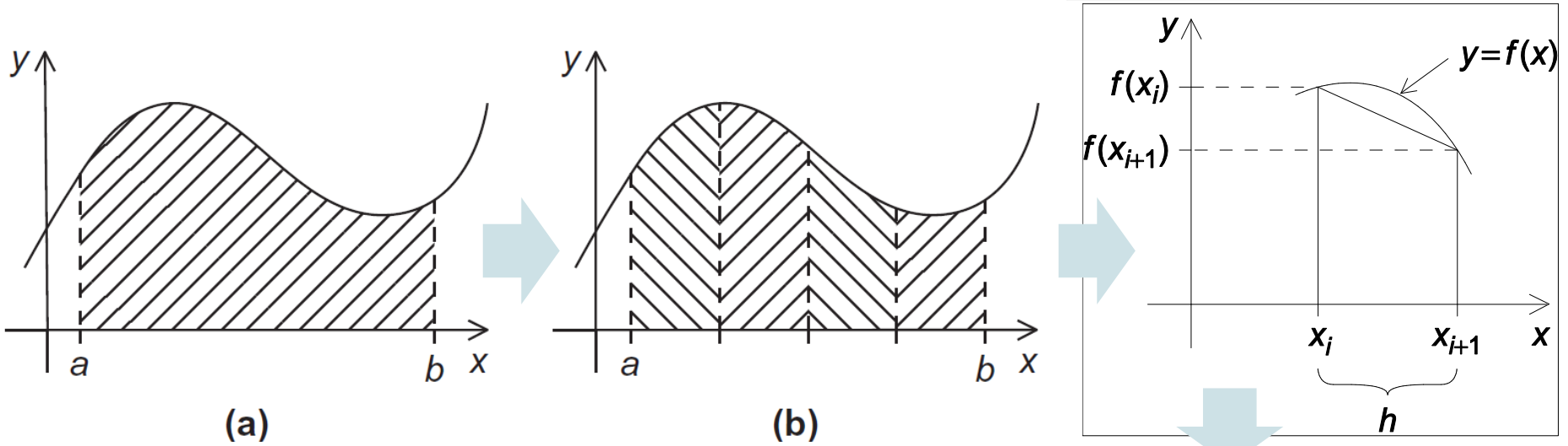
High Performance Computing for Data Science

MPI Communication
Lecture8 – 09/10/2026

The Trapezoidal rule in MPI

Prof. Sandro Fiore, Ph.D.
Department of Information Engineering and Computer Science (DISI)
University of Trento, 2026-2027

The Trapezoidal Rule (recap)



$$\text{Sum of trapezoid areas} = h[f(x_0)/2 + f(x_1) + f(x_2) + \cdots + f(x_{n-1}) + f(x_n)/2]$$

```
/* Input:  a, b, n */
h = (b-a)/n;
approx = (f(a) + f(b))/2.0;
for (i = 1; i <= n-1; i++) {
   x_i = a + i*h;
   approx += f(x_i);
}
approx = h*approx;
```

MORGAN KAUFMANN

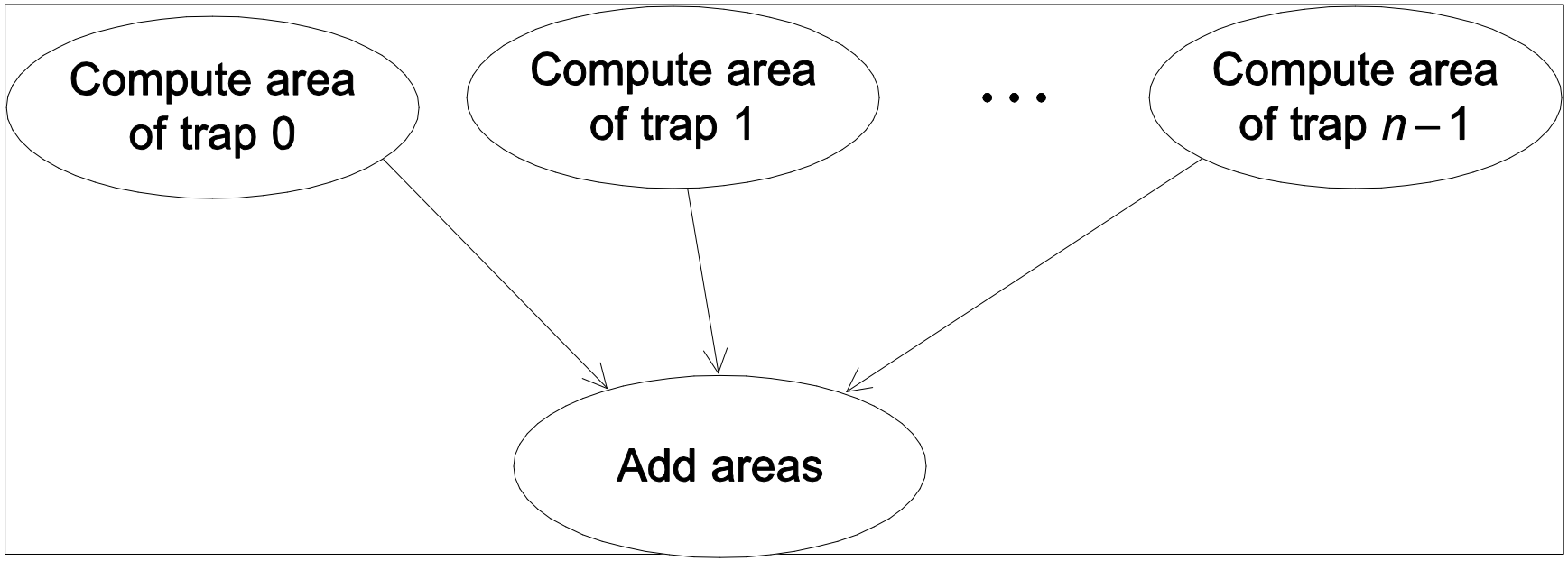
Copyright © 2010, Elsevier Inc. All rights Reserved

Parallelizing the Trapezoidal Rule

1. Partition problem solution into tasks.
2. Identify communication channels between tasks.
3. Aggregate tasks into composite tasks.
4. Map composite tasks to cores.

MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

Tasks and communications for Trapezoidal Rule



MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

Parallel pseudo-code

```
1     Get a, b, n;
2     h = (b-a)/n;
3     local_n = n/comm_sz;
4     local_a = a + my_rank*local_n*h;
5     local_b = local_a + local_n*h;
6     local_integral = Trap(local_a, local_b, local_n, h);
7     if (my_rank != 0)
8        Send local_integral to process 0;
9     else /* my_rank == 0 */
10       total_integral = local_integral;
11       for (proc = 1; proc < comm_sz; proc++) {
12          Receive local_integral from proc;
13          total_integral += local_integral;
14       }
15    }
16    if (my_rank == 0)
17       print result;
```

MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

First version (1)

```
1  int main(void) {
2     int my_rank, comm_sz, n = 1024, local_n;
3     double a = 0.0, b = 3.0, h, local_a, local_b;
4     double local_int, total_int;
5     int source;
6
7     MPI_Init(NULL, NULL);
8     MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);
9     MPI_Comm_size(MPI_COMM_WORLD, &comm_sz);
10
11    h = (b-a)/n;           /* h is the same for all processes */
12    local_n = n/comm_sz;   /* So is the number of trapezoids  */
13
14    local_a = a + my_rank*local_n*h;
15    local_b = local_a + local_n*h;
16    local_int = Trap(local_a, local_b, local_n, h);
17
18    if (my_rank != 0) {
19       MPI_Send(&local_int, 1, MPI_DOUBLE, 0, 0,
20             MPI_COMM_WORLD);
```

MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

First version (2)

```
21      } else {
22          total_int = local_int;
23          for (source = 1; source < comm_sz; source++) {
24              MPI_Recv(&local_int, 1, MPI_DOUBLE, source, 0,
25                  MPI_COMM_WORLD, MPI_STATUS_IGNORE);
26              total_int += local_int;
27          }
28      }
29
30      if (my_rank == 0) {
31          printf("With n = %d trapezoids, our estimate\n", n);
32          printf("of the integral from %f to %f = %.15e\n",
33              a, b, total_int);
34      }
35      MPI_Finalize();
36      return 0;
37  } /*  main  */
```

MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

First version (3)

```
/* Input:  a, b, n */
h = (b-a)/n;
approx = (f(a) + f(b))/2.0;
for (i = 1; i <= n-1; i++) {
   x_i = a + i*h;
   approx += f(x_i);
}
approx = h*approx;
```

```
1  double Trap(
2        double left_endpt  /* in */,
3        double right_endpt /* in */,
4        int    trap_count  /* in */,
5        double base_len    /* in */) {
6     double estimate, x;
7     int i;
8
9     estimate = (f(left_endpt) + f(right_endpt))/2.0;
10    for (i = 1; i <= trap_count-1; i++) {
11       x = left_endpt + i*base_len;
12       estimate += f(x);
13    }
14    estimate = estimate*base_len;
15
16    return estimate;
17 } /*  Trap  */
```

MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

Input

- Most MPI implementations only allow process 0 in MPI_COMM_WORLD access to stdin.
- Process 0 must read the data (scanf) and send to the other processes.

```
. . .
MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);
MPI_Comm_size(MPI_COMM_WORLD, &comm_sz);

Get_data(my_rank, comm_sz, &a, &b, &n);

h = (b-a)/n;
. . .
```

MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

Function for reading user input

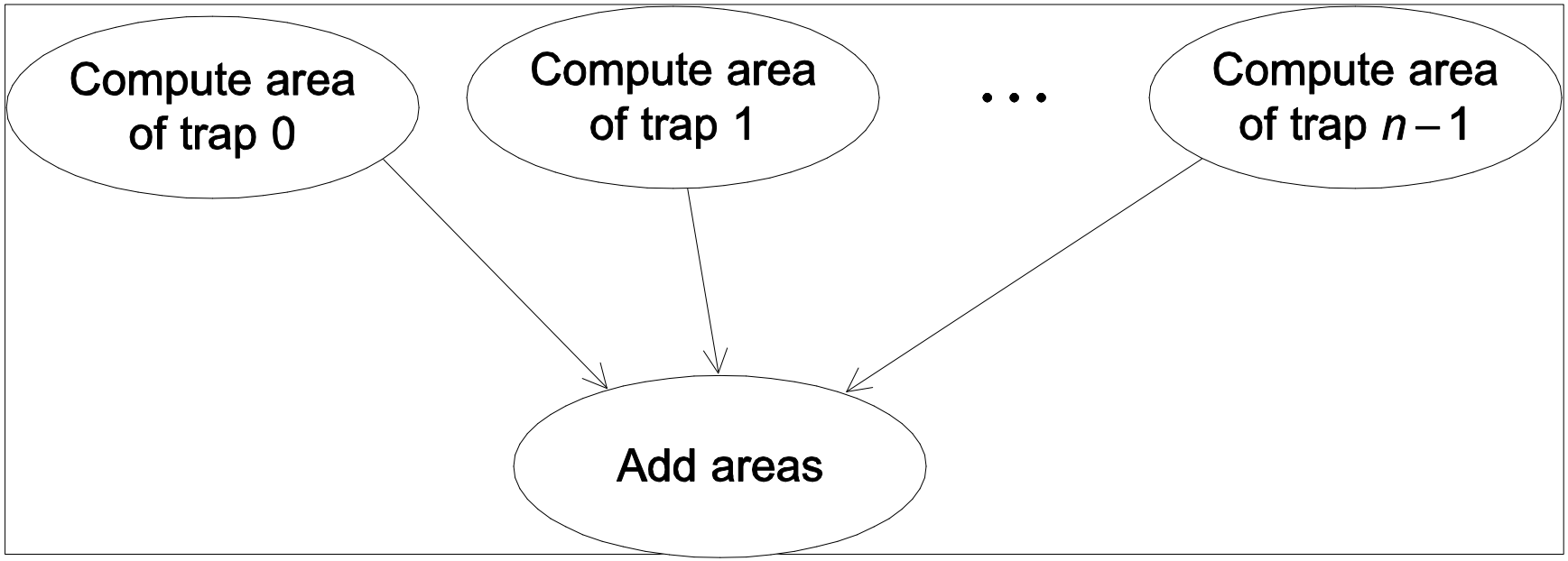
```
void Get_input(
      int       my_rank   /* in  */,
      int       comm_sz   /* in  */,
      double*   a_p       /* out */,
      double*   b_p       /* out */,
      int*      n_p       /* out */) {
   int dest;

   if (my_rank == 0) {
      printf("Enter a, b, and n\n");
      scanf("%lf %lf %d", a_p, b_p, n_p);
      for (dest = 1; dest < comm_sz; dest++) {
         MPI_Send(a_p, 1, MPI_DOUBLE, dest, 0, MPI_COMM_WORLD);
         MPI_Send(b_p, 1, MPI_DOUBLE, dest, 0, MPI_COMM_WORLD);
         MPI_Send(n_p, 1, MPI_INT, dest, 0, MPI_COMM_WORLD);
      }
   } else { /* my_rank != 0 */
      MPI_Recv(a_p, 1, MPI_DOUBLE, 0, 0, MPI_COMM_WORLD,
            MPI_STATUS_IGNORE);
      MPI_Recv(b_p, 1, MPI_DOUBLE, 0, 0, MPI_COMM_WORLD,
            MPI_STATUS_IGNORE);
      MPI_Recv(n_p, 1, MPI_INT, 0, 0, MPI_COMM_WORLD,
            MPI_STATUS_IGNORE);
   }
}  /* Get_input */
```

MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

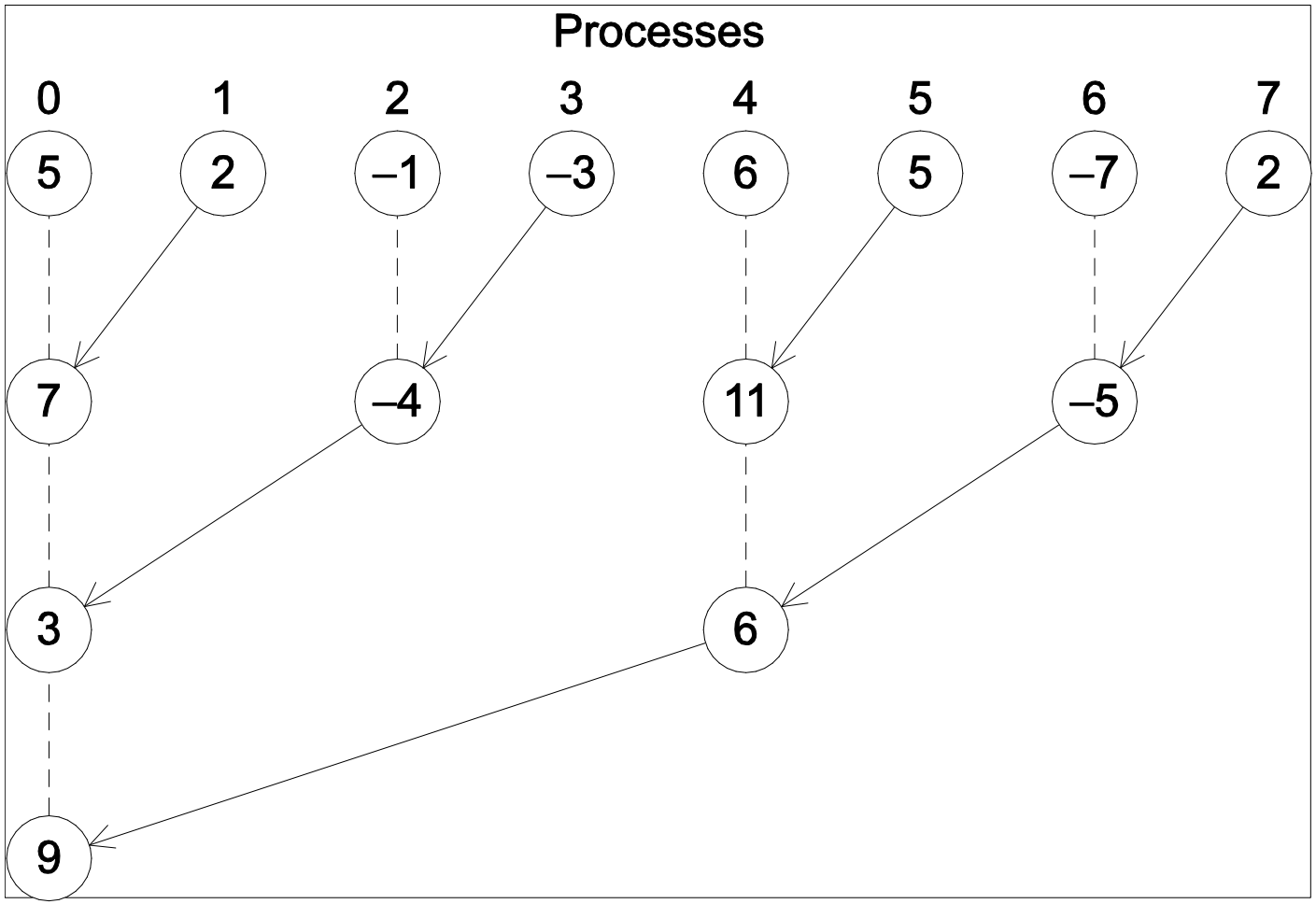
Tasks and communications for Trapezoidal Rule



Too much burden on the collector node
Can we better split the communication?

MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

A tree-structured global sum



MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

Tree-structured communication

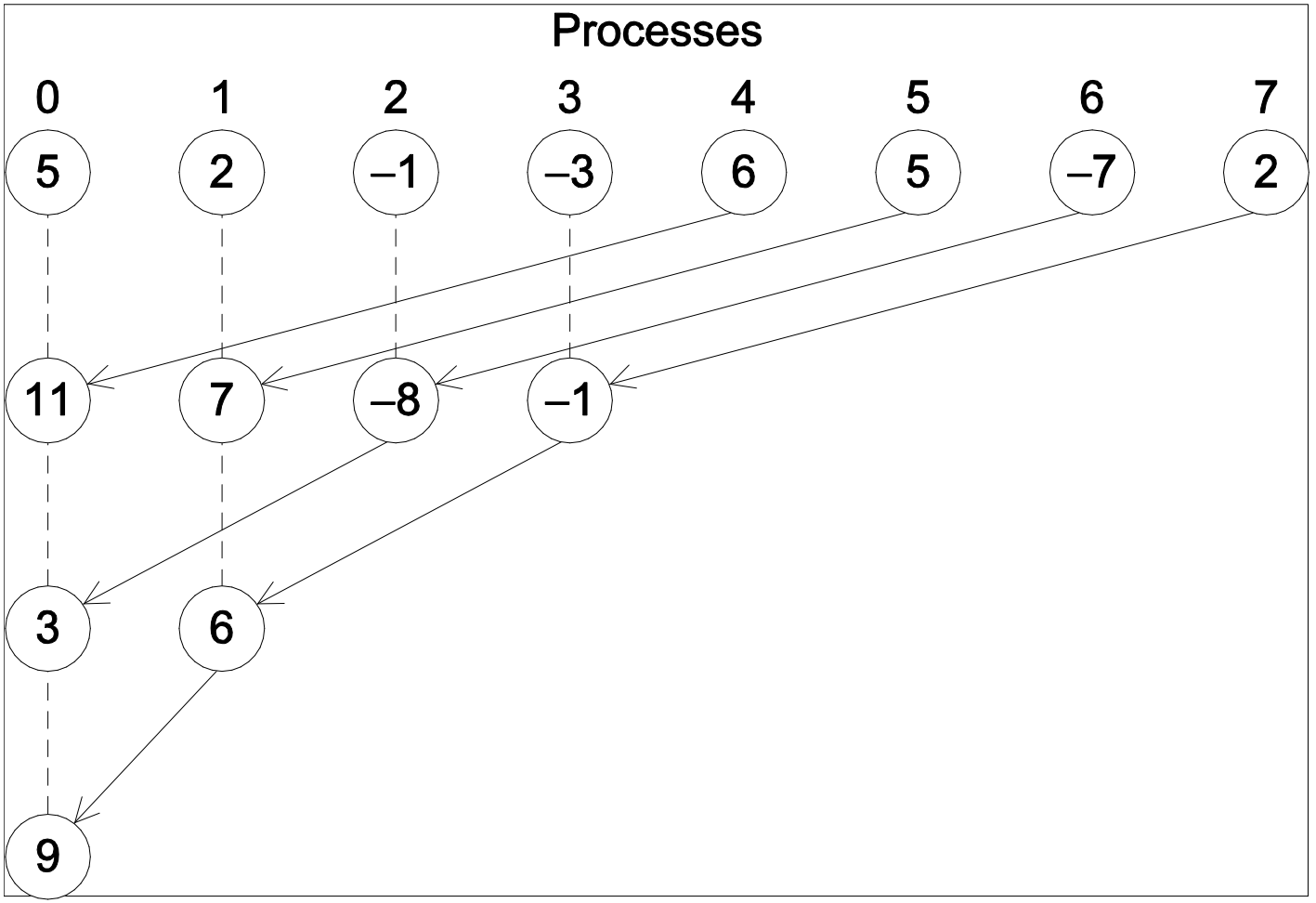
1. In the first phase:
   (a) Process 1 sends to 0, 3 sends to 2, 5 sends to 4, and 7 sends to 6.
   (b) Processes 0, 2, 4, and 6 add in the received values.
   (c) Processes 2 and 6 send their new values to processes 0 and 4, respectively.
   (d) Processes 0 and 4 add the received values into their new values.

2. (a) Process 4 sends its newest value to process 0.
   (b) Process 0 adds the received value to its newest value.

Is that the only way?

MK MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

An alternative tree-structured global sum



MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

MPI_Reduce

```
int MPI_Reduce(
      void*          input_data_p    /* in  */,
      void*          output_data_p   /* out */,
      int            count           /* in  */,
      MPI_Datatype   datatype        /* in  */,
      MPI_Op         operator        /* in  */,
      int            dest_process    /* in  */,
      MPI_Comm       comm            /* in  */);
```

```
MPI_Reduce(&local_int, &total_int, 1, MPI_DOUBLE, MPI_SUM, 0,
      MPI_COMM_WORLD);
```

```
double local_x[N], sum[N];
. . .
MPI_Reduce(local_x, sum, N, MPI_DOUBLE, MPI_SUM, 0,
      MPI_COMM_WORLD);
```

MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

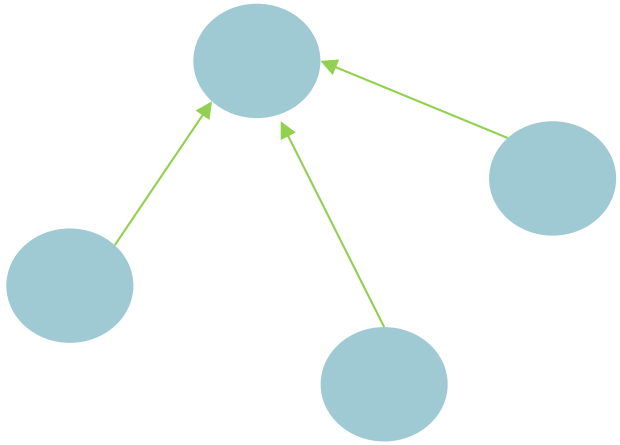
Predefined reduction operators in MPI

Operation Value	Meaning
MPI_MAX	Maximum
MPI_MIN	Minimum
MPI_SUM	Sum
MPI_PROD	Product
MPI_LAND	Logical and
MPI_BAND	Bitwise and
MPI_LOR	Logical or
MPI_BOR	Bitwise or
MPI_LXOR	Logical exclusive or
MPI_BXOR	Bitwise exclusive or
MPI_MAXLOC	Maximum and location of maximum
MPI_MINLOC	Minimum and location of minimum

MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

Communication hands-on

Implement a “reduce” by using send and receive + average/max/min over the set of data collected



Copyright © 2010, Elsevier Inc. All rights Reserved

Collective vs. Point-to-Point Communications

- All the processes in the communicator must call the same collective function.

- For example, a program that attempts to match a call to MPI_Reduce on one process with a call to MPI_Recv on another process is erroneous, and, in all likelihood, the program will hang or crash.

MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

Collective vs. Point-to-Point Communications

- The arguments passed by each process to an MPI collective communication must be "compatible."

- For example, if one process passes in 0 as the dest_process and another passes in 1, then the outcome of a call to MPI_Reduce is erroneous, and, once again, the program is likely to hang or crash.

Copyright © 2010, Elsevier Inc. All rights Reserved

Collective vs. Point-to-Point Communications

- The output_data_p argument is only used on dest_process

- However, all of the processes still need to pass in an actual argument corresponding to output_data_p, even if it's just NULL.

Copyright © 2010, Elsevier Inc. All rights Reserved

Collective vs. Point-to-Point Communications

- **Point-to-point** communications are matched on the basis of **tags** and **communicators**.
- **Collective** communications **don't use tags**.
- They're matched solely on the basis of the **communicator** and the **order** in which they're called.

Copyright © 2010, Elsevier Inc. All rights Reserved

Example (1)

Time	Process 0	Process 1	Process 2
0	`a = 1; c = 2`	`a = 1; c = 2`	`a = 1; c = 2`
1	`MPI_Reduce(&a, &b, ...)`	`MPI_Reduce(&c, &d, ...)`	`MPI_Reduce(&a, &b, ...)`
2	`MPI_Reduce(&c, &d, ...)`	`MPI_Reduce(&a, &b, ...)`	`MPI_Reduce(&c, &d, ...)`

Multiple calls to MPI_Reduce

Process 0 is acting as destination

What are the values in b and d for process 0?

MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

Example (2)

- Suppose that each process calls MPI_Reduce with operator MPI_SUM, and destination process 0.

- At first glance, it might seem that after the two calls to MPI_Reduce, the value of b will be 3, and the value of d will be 6.

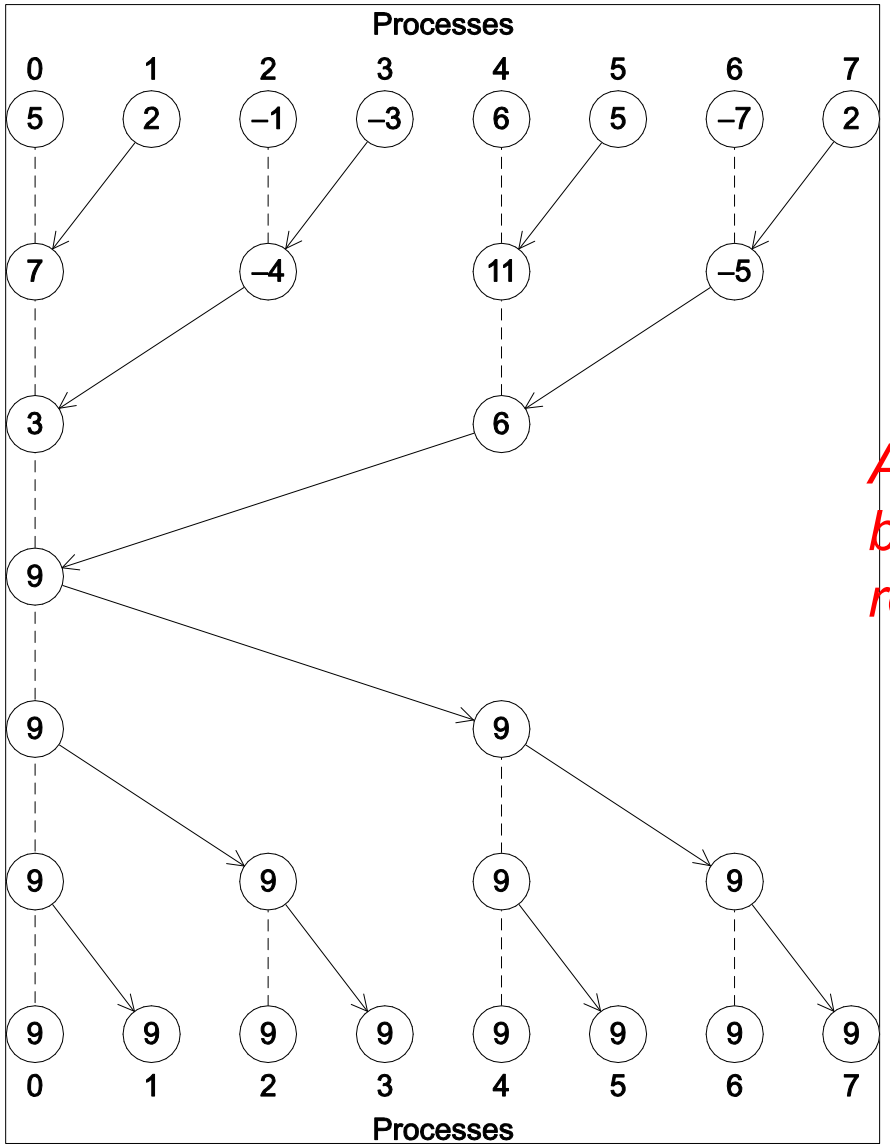
MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

MPI_Allreduce

- Useful in a situation in which all of the processes need the result of a global sum in order to complete some larger computation.

```
int MPI_Allreduce(
        void*        input_data_p   /* in  */,
        void*        output_data_p  /* out */,
        int          count          /* in  */,
        MPI_Datatype datatype       /* in  */,
        MPI_Op       operator       /* in  */,
        MPI_Comm     comm           /* in  */);
```

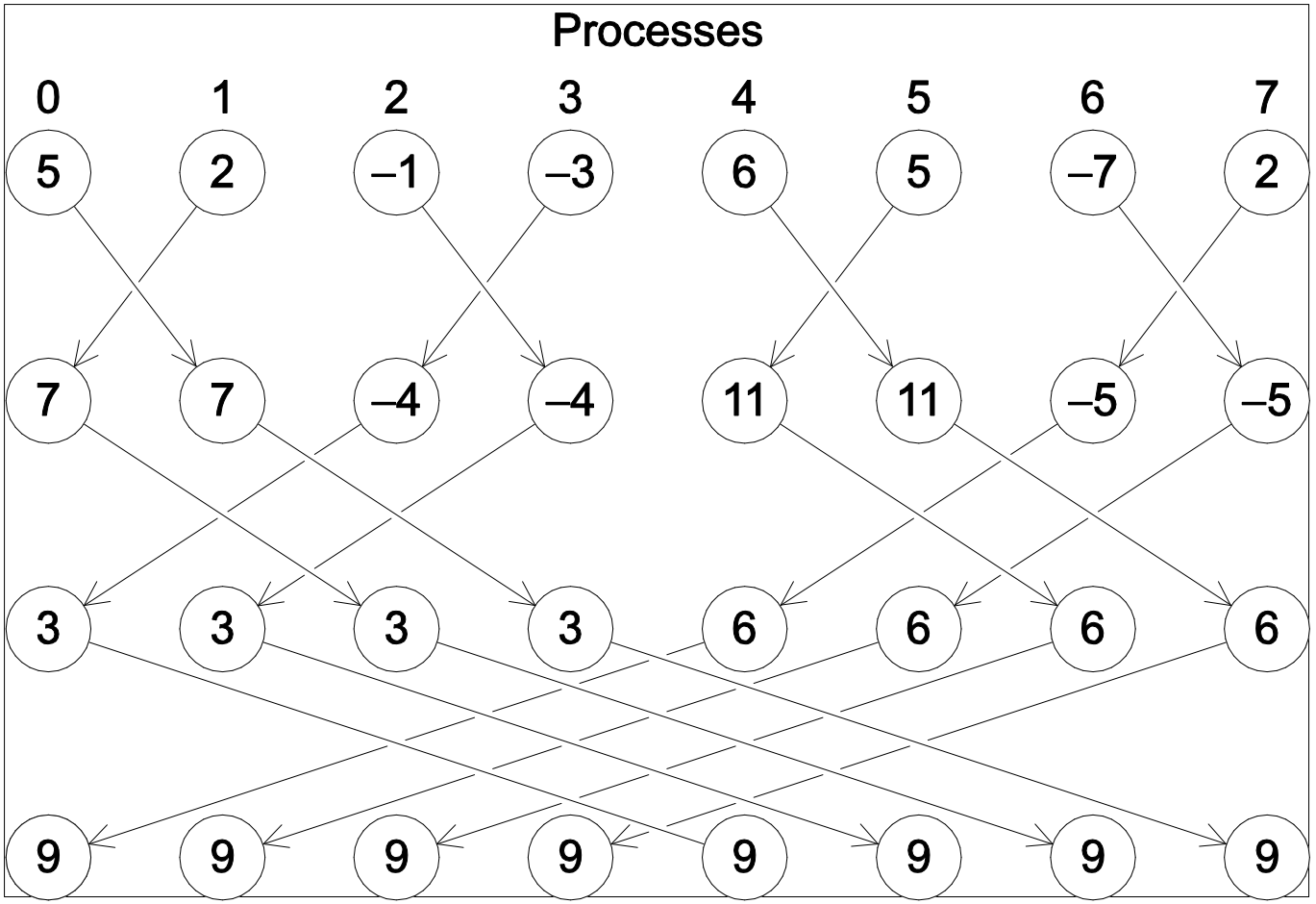
MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved



A global sum followed by distribution of the result.

MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved





A butterfly-structured global sum.

MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

Broadcast

- Data belonging to a single process is sent to all of the processes in the communicator.

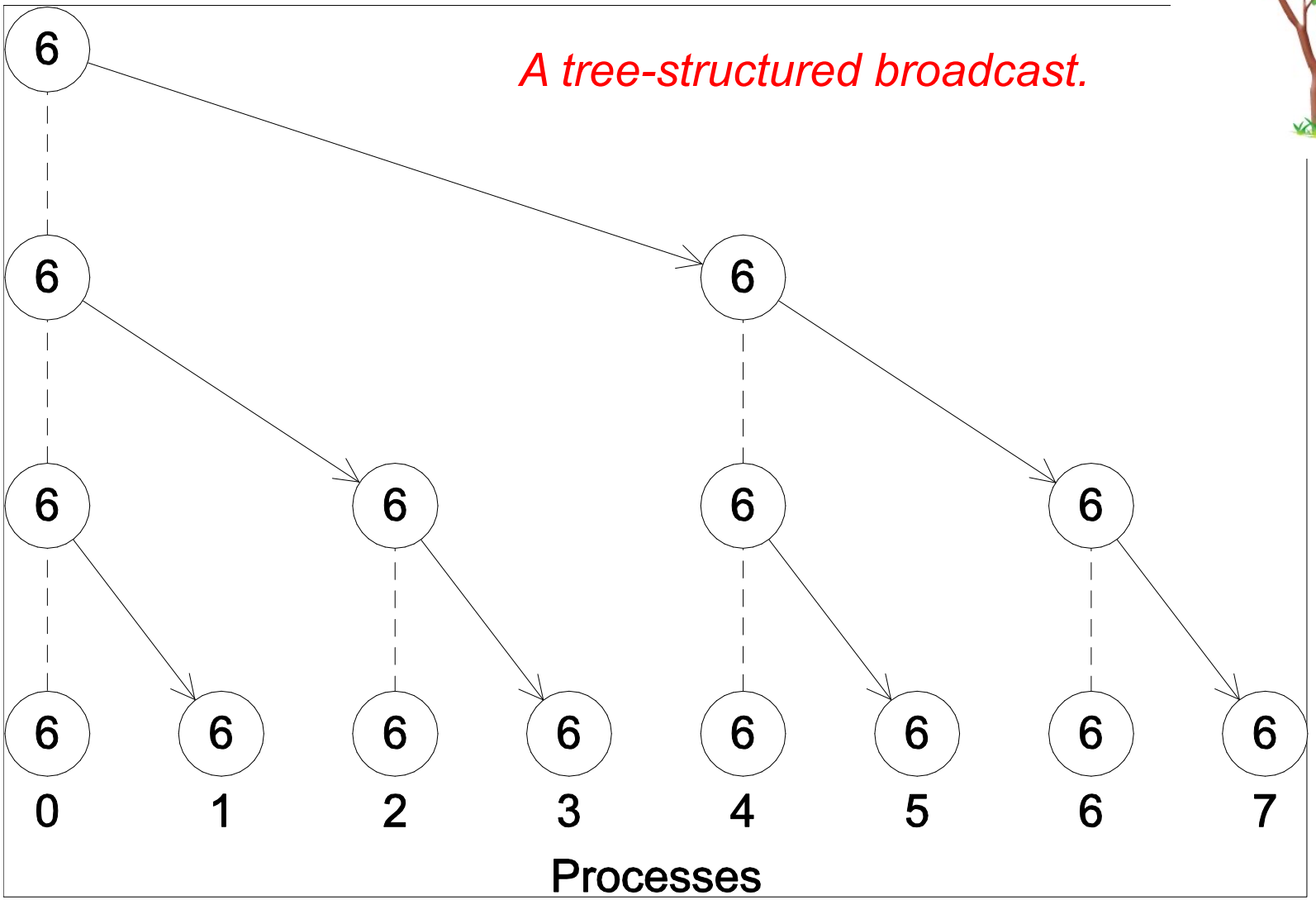
```
int MPI_Bcast(
      void*          data_p       /* in/out */,
      int            count        /* in     */,
      MPI_Datatype   datatype     /* in     */,
      int            source_proc  /* in     */,
      MPI_Comm       comm         /* in     */);
```

MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved



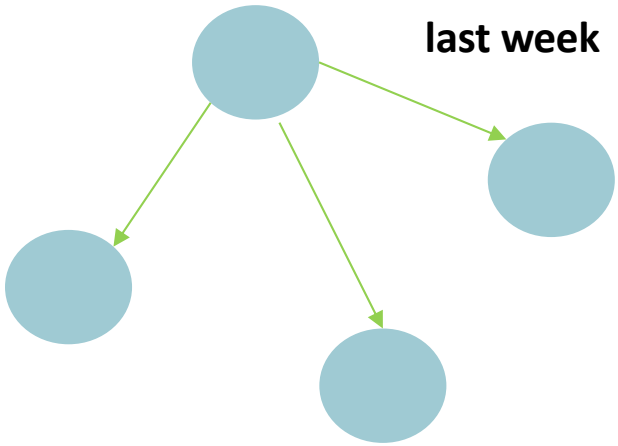
A tree-structured broadcast.



MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

Get_input without MPI_Bcast

Exercise last week



```
void Get_input(
      int       my_rank   /* in  */,
      int       comm_sz   /* in  */,
      double*   a_p       /* out */,
      double*   b_p       /* out */,
      int*      n_p       /* out */) {
   int dest;

   if (my_rank == 0) {
      printf("Enter a, b, and n\n");
      scanf("%lf %lf %d", a_p, b_p, n_p);
      for (dest = 1; dest < comm_sz; dest++) {
         MPI_Send(a_p, 1, MPI_DOUBLE, dest, 0, MPI_COMM_WORLD);
         MPI_Send(b_p, 1, MPI_DOUBLE, dest, 0, MPI_COMM_WORLD);
         MPI_Send(n_p, 1, MPI_INT, dest, 0, MPI_COMM_WORLD);
      }
   } else { /* my_rank != 0 */
      MPI_Recv(a_p, 1, MPI_DOUBLE, 0, 0, MPI_COMM_WORLD,
            MPI_STATUS_IGNORE);
      MPI_Recv(b_p, 1, MPI_DOUBLE, 0, 0, MPI_COMM_WORLD,
            MPI_STATUS_IGNORE);
      MPI_Recv(n_p, 1, MPI_INT, 0, 0, MPI_COMM_WORLD,
            MPI_STATUS_IGNORE);
   }
}  /* Get_input */
```

MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

Get_input that uses MPI_Bcast

```
void Get_input(
      int        my_rank   /* in  */,
      int        comm_sz   /* in  */,
      double*    a_p       /* out */,
      double*    b_p       /* out */,
      int*       n_p       /* out */) {

   if (my_rank == 0) {
      printf("Enter a, b, and n\n");
      scanf("%lf %lf %d", a_p, b_p, n_p);
   }
   MPI_Bcast(a_p, 1, MPI_DOUBLE, 0, MPI_COMM_WORLD);
   MPI_Bcast(b_p, 1, MPI_DOUBLE, 0, MPI_COMM_WORLD);
   MPI_Bcast(n_p, 1, MPI_INT, 0, MPI_COMM_WORLD);
}  /* Get_input */
```

MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

Feedback form

- Throughout the entire HPC4DS course there will be a form available for your feedback
  - Please provide any comment about:
    - Pros
    - Cons
    - Aspects that were not clear enough during the class
    - Any other feedback you think can be relevant for the course
    - …
- Information are gathered in an anonymous way

https://docs.google.com/forms/d/e/1FAIpQLScSVpNPfMP8poNNz-nLl9xlKlHkeBM2x_JgWyA0qQu6mOBl5A/viewform

Question time

Feedback form

- Throughout the entire HPC4DS course there will be a form available for your feedback
  - Please provide any comment about:
    - Pros
    - Cons
    - Aspects that were not clear enough during the class
    - Any other feedback you think can be relevant for the course
    - …
- Information are gathered in an anonymous way

https://docs.google.com/forms/d/e/1FAIpQLScSVpNPfMP8poNNz-nLl9xlKlHkeBM2x_JgWyA0qQu6mOBl5A/viewform